

[리액트] 글자 단위로 업데이트 로직을 구축하여 동시 편집 구현하기

1. 동시 편집 구현시 고려사항

동시 편집이 가능하게 되려면?

클라이언트가 **입력/삭제** 하는 대로 **글자 하나하나 실시간으로** 다른 클라이언트에게 반영되도록 해야한다.

우리의 프로젝트 Newcord에서는 게시글을 **블록 단위**로 구성하여 유저가 각각 작성 및 수정할 수 있도록 하였다.

현재 구현된

웹소켓을 통한 게시글(블록) 편집(업데이트) 관련 API는 아래와 같다.

```
1 {
2   "uuid": "testtest",
3   "userID" : 123123
4   "dto": {
5     "blockId": "66387520a4e82b361f7fd438",
6     "position" : 1,
7     "articleVersio" : "0.0",
8     "operationType" : "INSERT" //INSERT, DELETE, SEQUENCE_UPDATE
9     "entityType": "updatedType", //BLOCK, TEXT, TAG
10    "content": "updatedTestContent",
11    "updated_by": {
12      "updater_id": "testCreatorId",
13      "updated_at": "2007-12-03T10:15:30"
14    }
15  }
16 }
17
```

위의 코드는 API를 호출할 때의 Request Body이다.

위의 API를 **웹소켓**을 통해 클라이언트의 **변경사항을 글자 단위로** Send 하면

이를 다른 클라이언트들에게 Broadcast함으로써 반영하도록 한다.

여기서 우리가 주목해야 할 것은

`operationType : INSERT / DELETE`

이다.

유저가 입력할 때의 INSERT와 유저가 삭제할 때의 DELETE 두 가지를 공동 편집시에 주되게 사용할 예정이다.

그렇다면 여기서 중요한 건

 어떤 경우에 API를 호출해야 할까?

위의 질문을 고려하는 것에 어려움이 있었다.

단순히 삽입, 삭제의 개념이 아니라 여러 가지 경우의 수를 고려하여 API를 호출하는 시점을 결정해야 했다.

ex) 삽입이 발생했다면 삽입된 글자만을 해당 블록의 삽입 위치에 이전에 있던 앞과 뒤의 문자열은 그대로 뱉두고 그 사이에 글자를 삽입해야 한다.

2. 초기 로직 구현 방식 [🔗](#)

초기에는 **글자 수**에 따라 판단하려고 했다.

입력하고 있는 문자열과 입력 되어있던 문자열의 길이가 다를 때

→ 입력하고 있는 문자열이 더 길면? 늘어난 인덱스 하나만 INSERT

→ 더 짧으면? 줄어든 인덱스 하나만 DELETE

(더 복잡한 로직이 필요했지만 일단은 이런식으로만 판단해야겠다고 생각했다... 당시 내 최선이였다...)

하지만 문제가 발생했다.

이런 식으로 한다면 **붙여넣기**(한번에 두 개 이상의 변경사항 발생)에는 제대로 작동을 하지 못한다.

(하나의 인덱스만 업데이트 할 수 있기 때문)

3. 최종 구현 로직 [🔗](#)

결론적으로 내가 고려해야할 사항들은 아래와 같은 두 가지였다.

- 어디가 바뀌었는가? (position)
- 얼마나 바뀌었는가? (content)

이를 기반으로 로직을 구축해보았다.

먼저 INSERT 와 DELETE가 발생했을 때

API를 호출할 때에 필요한 정보를 구하는 **함수**를 구현했다.

[Insert] [🔗](#)

```
1 function findInsertionIndices(original, modified) {
2   let startIndex = 0;
3   let endIndexOriginal = original.length;
4   let endIndexModified = modified.length;
5
6   // Find the start index where the original and modified first differ
7   while (
8     startIndex < endIndexOriginal &&
9     startIndex < endIndexModified &&
10    original[startIndex] === modified[startIndex]
11  ) {
12    startIndex++;
13  }
14
15  // Find the end index where the original and modified last differ
16  while (
17    endIndexOriginal > startIndex &&
18    endIndexModified > startIndex &&
19    original[endIndexOriginal - 1] === modified[endIndexModified - 1]
20  ) {
21    endIndexOriginal--;
22    endIndexModified--;
23  }
24
25  return {
```

```

26     startIndex: startIndex,
27     endIndex: endIndexModified,
28   };
29 }

```

- insert가 발생했을 때 이전 문자열과 변화한 문자열을 비교하여 변경된 부분의 처음 인덱스와 마지막 인덱스를 찾는 함수이다.
- 문자열간 비교를 할 때 변경된 부분을 탐색하는 것에 있어 [중복도 고려](#)하도록 하였다.

[Delete] [🔗](#)

```

1 function findDeletionIndices(original, modified) {
2   let startIndex = 0;
3   let endIndexOriginal = original.length;
4   let endIndexModified = modified.length;
5
6   // Find the start index where the original and modified first differ
7   while (
8     startIndex < endIndexOriginal &&
9     startIndex < endIndexModified &&
10    original[startIndex] === modified[startIndex]
11  ) {
12    startIndex++;
13  }
14
15  // Find the end index where the original and modified last differ
16  while (
17    endIndexOriginal > startIndex &&
18    endIndexModified > startIndex &&
19    original[endIndexOriginal - 1] === modified[endIndexModified - 1]
20  ) {
21    endIndexOriginal--;
22    endIndexModified--;
23  }
24
25  return {
26    startIndex: startIndex,
27    endIndex: endIndexOriginal,
28  };
29 }

```

- 마찬가지로 이전 문자열과 현재 문자열을 비교하여 삭제된 내용의 처음 인덱스와 마지막 인덱스를 구한다.

위의 두 함수를 이용하여 바뀐 글자의 내용을 API를 통해 전달 할 수 있도록 되었고 다른 유저들은 이러한 내용들을 receive하여 반영할 수 있다.

최종 로직 [🔗](#)

이후 데모에서 실시간으로 잘 반영되는 것을 볼 수 있다.

```

1 if (afterContent.length > beforeContent.length) {
2   const changed = findInsertionIndices(beforeContent, afterContent);
3
4   updateBlock(
5     'INSERT',
6     blockIds[editingIndex],
7     changed.startIndex,
8     afterContent.slice(changed.startIndex, changed.endIndex)

```

```
9      );
10     return;
11 } else if (beforeContent.length > afterContent.length) {
12     const deleted = findDeletionIndices(beforeContent, afterContent);
13
14     updateBlock(
15         'DELETE',
16         blockIds[editingIndex],
17         deleted.startIndex,
18         deleted.endIndex - deleted.startIndex
19     );
20 }
```